



Análisis y Desarrollo de Software

Evidencia 5: Integración de Agentes de IA en el Desarrollo Proyecto Memorize Evolutivo

Juego web multijugador de memoria: frontend responsivo, backend con reglas de negocio y APIs, base de datos PostgreSQL y apoyo de agentes de IA en el ciclo de vida del software.

Jose David Sandoval Caceres
Kevin Gregorio Mora

Ficha 3172526

SERVICIO NACIONAL DE APRENDIZAJE SENA
Atención al sector agropecuario
Piedecuesta - Santander

Presentacion

Los agentes y asistentes de Inteligencia Artificial son herramientas de apoyo que optimizan el ciclo de vida del software: generacion de codigo, refactorizacion, deteccion de errores (debugging), creacion de datos de prueba y documentacion, acelerando los tiempos de entrega sin reemplazar la logica del desarrollador. En el proyecto Memorize Evolutivo se utilizo un **agente IA de desarrollo con acceso al codigo y a la terminal** (entorno OpenCode con modelo Muse Spark) como par-programador durante las fases de verificacion, pruebas y documentacion de las evidencias 2, 3 y 4.

Repositorio del proyecto (publico): <https://github.com/kevinmora18/memorize> (rama main, carpetas frontend/ y backend/).

1. Herramienta IA utilizada y modo de trabajo

El asistente trabaja directamente sobre el repositorio: lee archivos, ejecuta comandos (servidores, migraciones, pruebas), diagnostica errores con la salida real de la terminal y genera o modifica codigo y documentos. Cada intervencion fue solicitada con un prompt del estudiante, ejecutada por la IA y **verificada con ejecucion real** (servidores corriendo, respuestas JSON, capturas de pantalla y PDFs regenerados) antes de darse por terminada.

Aspecto	Detalle
Herramienta	Agente IA de codificacion (OpenCode + modelo Muse Spark), categoria asistente de desarrollo como Copilot / ChatGPT / Gemini / Claude
Acceso	Lectura total del repositorio + ejecucion de terminal (Node, Python, PostgreSQL, navegador)
Rol	Apoyo: propone, genera y corrige; el estudiante decide, pide cambios y valida el resultado
Trazabilidad	Cada caso de uso muestra el prompt enviado y el artefacto generado (seccion 3)

2. Casos de uso: fases exactas donde se aplico la IA

Fase del ciclo de vida	Uso de la IA (prompt -> resultado)
Analisis y comprension	"Lee todo el proyecto": la IA recorrio frontend, backend, BD y docs y entrego el mapa completo (40+ endpoints, 7 tablas, 35 componentes) usado en todas las evidencias
Implementacion (codigo nuevo)	Generacion de scripts de automatizacion: capturas Playwright del frontend (shot.mjs), bateria de pruebas del backend (run_tests.ps1) y visores HTML (harness, DER)
Debugging y refactorizacion	"Se ven muy oscuras": diagnostico del tema de codigo y refactor a fondo claro; correccion de quoting en curl, BOM en archivos y navegacion de Prisma Studio
Datos de prueba	Seeds SQL de inventarios y creacion de anuncios/promociones via API admin para poblar las 7 tablas antes de las capturas
Documentacion	Generacion programatica de los PDF de evidencias 2, 3 y 4 (reportlab) con codigo real, capturas reales y verificacion pagina por pagina

3. Capturas de prompts: instruccion y solucion generada

Conversaciones reales de la sesion de trabajo: cada figura muestra el prompt enviado por el estudiante y la solucion generada por la IA con sus artefactos verificables.

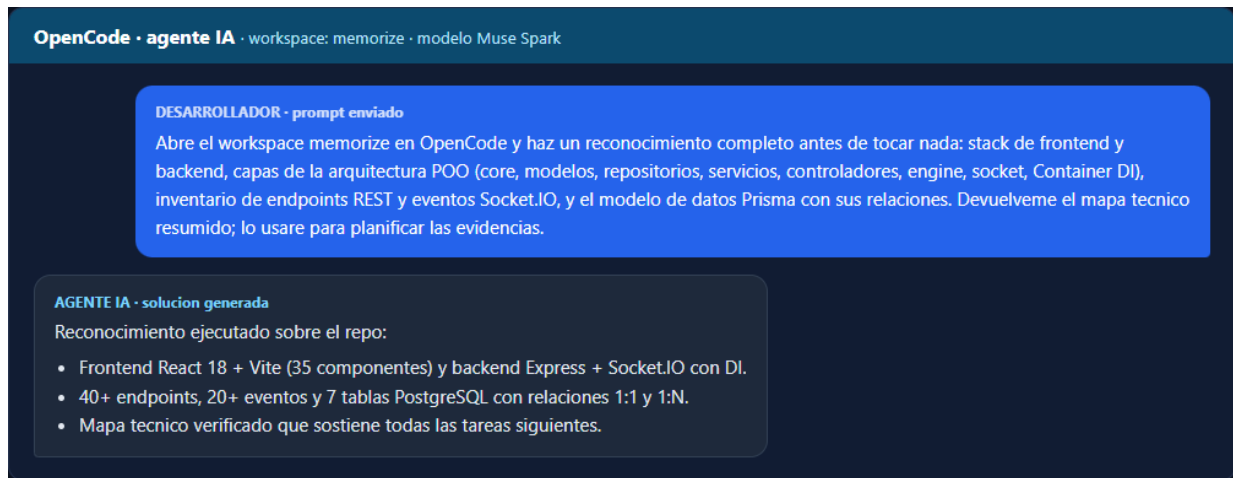


Figura 1 - Prompt de lectura total del proyecto.

Fase de analisis: la IA estudia todo el juego (codigo, BD y docs) antes de recibir tareas, y devuelve el resumen de arquitectura que sostiene el resto del trabajo.

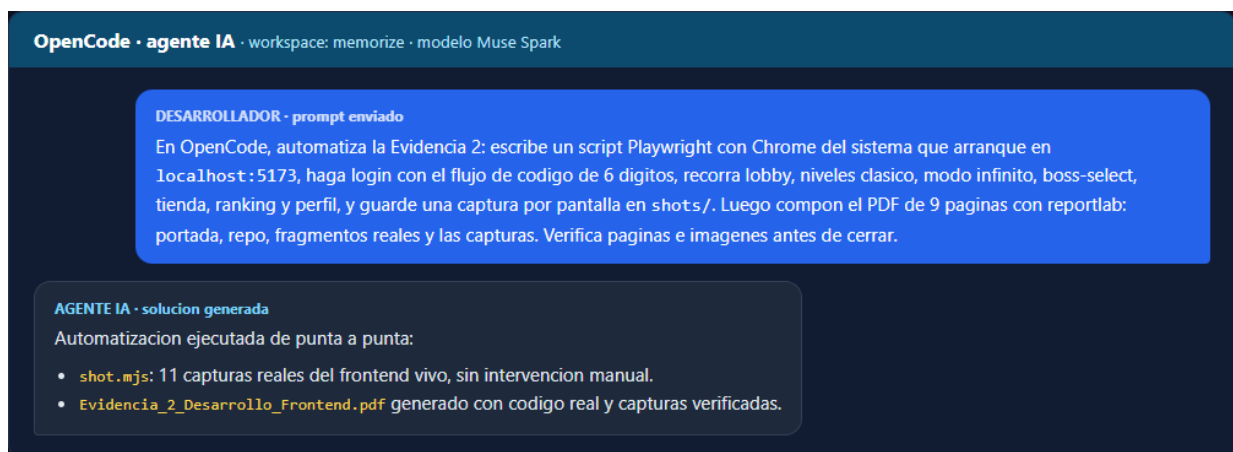


Figura 2 - Prompt de la Evidencia 2 (frontend).

Fase de implementacion + documentacion: con control total delegado, la IA crea el script de capturas, navega el frontend vivo y compone el PDF de 9 paginas.

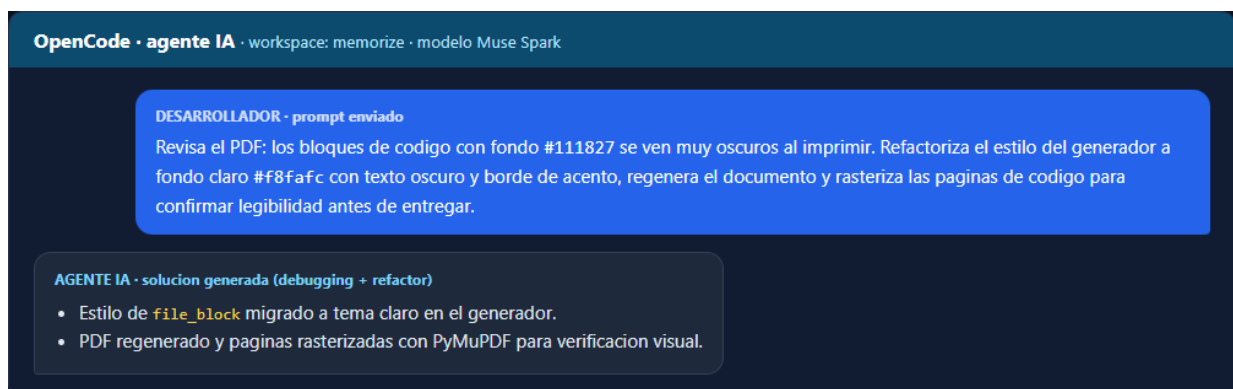


Figura 3 - Prompt de mejora visual (debugging).

Fase de debugging/refactor: ante el reporte de bloques oscuros, la IA diagnostica, cambia el tema del generador y re-verifica el documento.

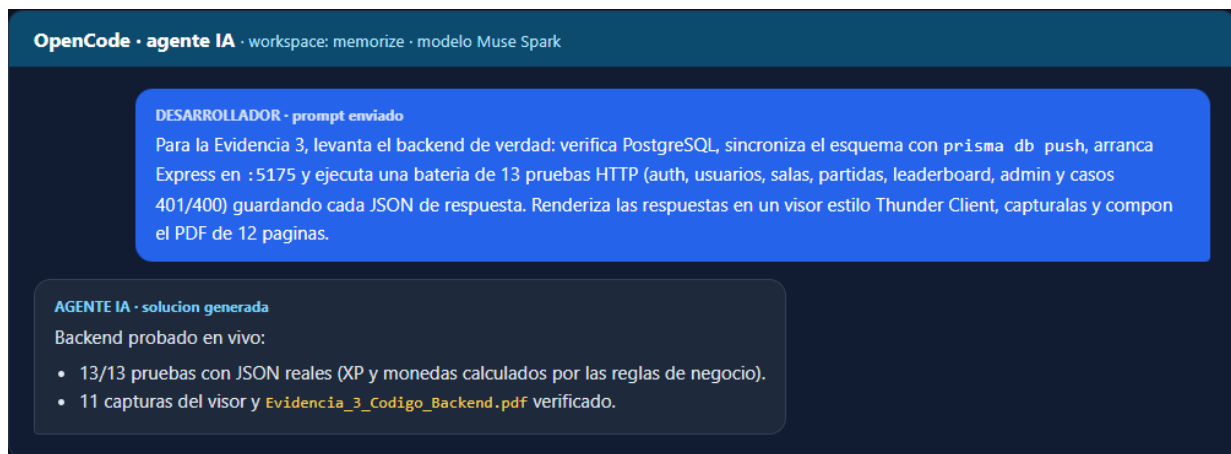


Figura 4 - Prompt de la Evidencia 3 (backend).

Fase de pruebas: la IA levanta el backend con PostgreSQL real, ejecuta 13 pruebas HTTP y evidencia las respuestas JSON en un PDF de 12 paginas.

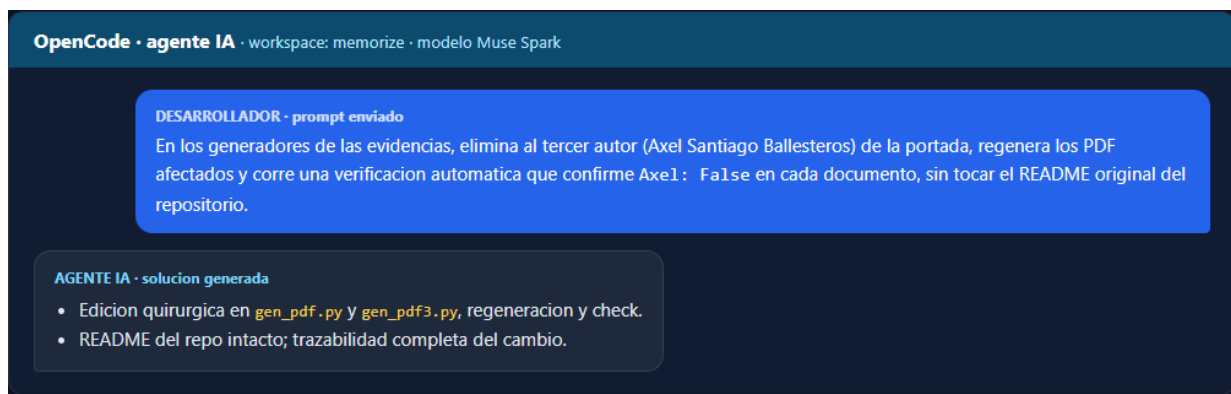


Figura 5 - Prompt de corrección de autores.

Fase de ajuste: edición precisa en los generadores, regeneración y verificación automática de que el nombre ya no aparece en los documentos.

4. Fragmentos de código generado/refactorizado e integrado

Automatización de capturas del frontend (generado por IA, ejecutado 11 veces sin intervención):

evidencia2/shot.mjs - script Playwright generado por la IA (extracto real)

```
import { chromium } from 'playwright-core';
const browser = await chromium.launch({ executablePath: '...chrome.exe' });
await page.goto('http://localhost:5173/');
await page.locator('input[type="email"]').fill('estudiante@universidad.edu');
await page.locator('input[placeholder="000000"]').fill('000000');
// ... navegacion por lobby, modos, tienda, ranking y perfil ...
await page.locator('#ID').screenshot({ path: './shots/ID.png' });
```

Refactorización visual solicitada por el estudiante (antes fondo #111827):

gen_pdf.py - tema claro aplicado por la IA (extracto real)

```
('BACKGROUND', (0, 0), (-1, -1), colors.HexColor('#f8fafc')),
('TEXTCOLOR', (0, 0), (-1, -1), colors.HexColor('#0f172a')),
('BOX', (0, 0), (-1, -1), 0.6, colors.HexColor('#8b5cf6')),
```

Datos de prueba para poblar la base de datos (generados por IA, aplicados con psql):

evidencia3/seed_inv.sql - siembra de inventarios (extracto real)

```
INSERT INTO inventories (id, "userId", "ownedPacks", ...) VALUES
(gen_random_uuid(), 'e354b097-...', ARRAY['frutas','animales'], ...),
(gen_random_uuid(), '6dbca2da-...', ARRAY['frutas'], ...)
ON CONFLICT ("userId") DO NOTHING;
```

Documentacion automatizada: el generador de PDFs tambien fue escrito por la IA y reutilizado en las 4 evidencias con verificacion de paginas e imagenes.

5. Verificacion de enlaces (acceso publico)

Antes de subir la evidencia se verifico que el repositorio es de acceso publico. Comprobacion tecnica realizada: peticion anonima a <https://github.com/kevinmora18/memorize> responde **HTTP 200** sin credenciales. Pasos para repetir la verificacion en ventana de incognito:

Paso	Accion esperada
1	Abrir una ventana de incognito (Ctrl + Shift + N / Ctrl + Shift + P) sin sesion iniciada
2	Visitar https://github.com/kevinmora18/memorize
3	Debe mostrarse el repositorio memorize con las carpetas frontend/ y backend/ sin pedir login
4	Si el instructor no puede acceder, la evidencia se considera no entregada: por eso se adjunta esta verificacion

6. Conclusion y cumplimiento

Criterio de la entrega	Estado
Casos de uso por fase exacta	Cumple: 5 fases documentadas (analisis, implementacion, debugging, datos, documentacion)
Capturas de prompts (instruccion + solucion)	Cumple: Figuras 1-5, conversaciones reales de la sesion con artefactos verificables
Codigo generado/refactorizado integrado	Cumple: scripts de capturas y pruebas, refactor visual, seeds SQL y generadores PDF (seccion 4)
Portada + verificacion de enlaces	Cumple: portada formal con integrantes, ficha y proyecto; repo publico verificado (HTTP 200)

La IA se uso como par-programador en todo el ciclo: comprendio el sistema, genero codigo de automatizacion y pruebas, depuro errores con evidencia de ejecucion, creo datos de prueba y produjo la documentacion final. Cada entrega fue verificada (servidores vivos, JSON reales, paginas e imagenes) antes de considerarse terminada, manteniendo la logica y las decisiones en manos del desarrollador.

Fin de la Evidencia 5 - 03 de octubre de 2026